OS Orange -4 Lab Report: PES-VCS

Name: Ruturaj R S
SRN: PES1UG24CS606
Assignment: OS Orange -4
Platform: Ubuntu 22.04

1. Objective

This assignment implements a local version control system (PES-VCS) with:

- content-addressable object storage,
- tree snapshot construction,
- staging index management,
- commit creation and history traversal,
- verification through unit and integration tests.

2. Implementation Summary

Implemented all required TODO functions:

- object.c
  - object_write
  - object_read
- tree.c
  - tree_from_index
- index.c
  - index_load
  - index_save
  - index_add
- commit.c
  - commit_create

3. Screenshot Evidence

1A. Phase 1 Unit Test (./test_objects)

```
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$
Explore and understand your code
```

1B. Sharded Object Storage (find .pes/objects -type f)

```
All Phase 1 tests passed.
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$
Explore and understand your code
```

1B Sharded object files

2A. Phase 2 Unit Test (./test_tree)

```
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$ make test_tree
./test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$
Explore and understand your code
```

2A Phase 2 test_tree

2B. Raw Tree Object Binary Format (xxd … | head -20)

```
xxd: .pes/objects/xx/rrr: No such file or directory
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$ xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476 | head -20
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$
```

2B Raw tree xxd

3A. pes init + pes add + pes status

```
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$ make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_objects
  untracked:  commit.h
  untracked:  .gitignore
  untracked:  test_sequence.sh
  untracked:  object.c
  untracked:  test_objects.c
  untracked:  pes.c
  untracked:  README.md
  untracked:  commit.c
  untracked:  index.h
  untracked:  tree.c
  untracked:  test_tree
  untracked:  tree.h
  untracked:  test_tree.c
  untracked:  Makefile
  untracked:  pes.h
  untracked:  index.c
  untracked:  output.txt

ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$
```

3A Phase 3 init add status

3B. Index Text Format (cat .pes/index)

```
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776425916 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776425916 6 file2.txt
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$
```

3B Phase 3 index text

4A. Commit History (./pes log with three commits)

4B. Object Store Growth (find .pes -type f | sort)

```
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: ac543c0c6395... Initial commit
Committed: 85e259207939... Add world
Committed: b784817abb0e... Add farewell
commit b784817abb0e72f7d0f44ade4d5cff1fbe36fa0e8951343cd1b911e8fd917ff8
Author: PES User <pes@localhost>
Date:   1776426562

    Add farewell

commit 85e2592079394be3d748d6237606d70458f99e3c691fc977a6199e4b5baed482
Author: PES User <pes@localhost>
Date:   1776426562

    Add world
```

4C. Reference Chain (cat .pes/refs/heads/main and cat .pes/HEAD)

```
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/00/c2b1a0d9f83a3aa23b77778f810b91968570d2015ab81a54d001999716c240
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/0e/ef253b9e728dc0b08319c8f9979a579b67ca6ce11feee29aa2f39105b72439
.pes/objects/1b/c54e74cc64ae225f18b3f79b939933c593b871807204dbb9c7a696d5336209
.pes/objects/3a/8d84cf442d1533eb09fc0673ee76fe23faea18657dd17d98bd4305ca2273a3
.pes/objects/3d/c0e0d16aa52d9f088f194863711266d726bbc2203a48b86fc204cd67ae6574
.pes/objects/45/1f725d0b7ca0d4753c49884262a0275a6ec9a49b339364d8928cf4c77ec29c
.pes/objects/46/bd4e8ed24d37d91bd1c7f91c629c2f6c47e75faaa9c5fb1ae544bed17e7c51
.pes/objects/49/c0927a96ad4248282ef815933d7be7e8e44e3ef3390a79c37ecac89f1b6372
.pes/objects/49/f7a714134a734b5dec7bf0abf467bde4a43d45fcdc3d4d0261d92484c0c098
.pes/objects/53/16bbe8b982120845f1c563b09d299942ff4875da88905f0705f558eac1cade
.pes/objects/60/031a1d6a8c96a3aa1570dd76ac9ea367784de61f22e9769653c50ff65b3705
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/85/e2592079394be3d748d6237606d70458f99e3c691fc977a6199e4b5baed482
.pes/objects/8b/cfe6dacce4e1442c5bca106b53408e9287759c75874c9047c8e781fe0fb0a2
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/a7/47c6a5f697fc60110af1d11598a54f88205dd68ddf44082f15718430f55abe
.pes/objects/ac/543c0c6395f389c8e662611d13c9f0cf24ceb34757d042809abb458ee3a5a2
.pes/objects/b2/30bcc961573a966fd35c62c4d39d3dffb2481d6ea0985049625aaaa4c70614
.pes/objects/b7/84817abb0e72f7d0f44ade4d5cff1fbe36fa0e8951343cd1b911e8fd917ff8
.pes/objects/bb/c771a1ea54b3d6d8e7d06482c6a3eb33002db7cd3591f82e06f0589b3e1299
.pes/objects/bd/85e61a13fd1120ab7a37bb97b2c7ee0af9ff876c1ed4b058f0e5e856c20766
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/c1/52aeb0b90836a6af4bd168c577996b55e46e55ac3704489b88467fb13a3edb
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ef/2285ea45a5c516a624ace54fe6d33c7aded897e8b8491de27d65a67499e98a
.pes/objects/fa/ee676b0aad726aff3c1a317f874a9fca8ac90e6d2fc87939445c682125af5b
.pes/refs/heads/main
```

4C Phase 4 refs and HEAD

Final Integration Test (make test-integration)

```
ruturaj@ruturaj-LOQ:~/PES1UG24CS606-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 09c4d4674ac6... Initial commit

Log after first commit:
commit 09c4d4674ac63f191df8e40c81a7d0314cd763c2c7a76057a0b6f6c8fe4de1a7
Author: PES User <pes@localhost>
Date:   1776426804

    Initial commit


--- Second Commit ---
Committed: f207f30651f6... Update file.txt

--- Third Commit ---
Committed: f70b2d730aef... Add farewell

--- Full History ---
commit f70b2d730aef0afb8a3508f030d33d8cca9ec6cd8350836f5f0b7a1a7353c91d
Author: PES User <pes@localhost>
Date:   1776426804

    Add farewell
```

```
    Add farewell

commit f207f30651f604074b8b7dcf9f93bac21ad83e0210cbf39a5a9e1762c8646b05
Author: PES User <pes@localhost>
Date:   1776426804

    Update file.txt

commit 09c4d4674ac63f191df8e40c81a7d0314cd763c2c7a76057a0b6f6c8fe4de1a7
Author: PES User <pes@localhost>
Date:   1776426804

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
f70b2d730aef0afb8a3508f030d33d8cca9ec6cd8350836f5f0b7a1a7353c91d

--- Object Store ---
Objects created:
10
.pes/objects/09/c4d4674ac63f191df8e40c81a7d0314cd763c2c7a76057a0b6f6c8fe4de1a7
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f2/07f30651f604074b8b7dcf9f93bac21ad83e0210cbf39a5a9e1762c8646b05
.pes/objects/f7/0b2d730aef0afb8a3508f030d33d8cca9ec6cd8350836f5f0b7a1a7353c91d

=== All integration tests completed ===
```

4. Analysis Answers

Q5.1: How would pes checkout work?

A branch is a ref file in .pes/refs/heads storing a commit hash.

To implement checkout:

1. Resolve the target commit hash from .pes/refs/heads/.
2. Update .pes/HEAD to point to ref: refs/heads/ (or hash for detached checkout).
3. Read target commit -> target tree.
4. Rewrite working directory to match target tree:
    - create/update files from blobs,
    - create/remove directories,
    - apply modes (100644 or 100755),
    - remove tracked paths not present in target tree.
5. Rewrite .pes/index so staged state matches checked-out snapshot.

Why complex:

- Must safely coordinate HEAD, index, and working directory updates.
- Must avoid overwriting local uncommitted work.
- Must handle deletes, nested trees, and mode changes atomically.

Q5.2: Detect dirty working directory conflicts using index + object store

Use three snapshots:

- A: index entries (path -> staged blob hash + metadata)
- B: current HEAD tree (from object store)
- C: target branch tree (from object store)

Conflict detection:

1. For each tracked path in union(A, B), inspect working directory file.
2. If missing or metadata differs from index entry, confirm by hashing file and comparing with A[path].hash.
3. Mark path dirty if working file != staged/index version.
4. If dirty path would change between current and target (B[path] != C[path], or present/deleted mismatch), refuse checkout.

This prevents local changes from being overwritten by branch switching.

Q5.3: Detached HEAD behavior and recovery

Detached HEAD means .pes/HEAD stores a commit hash directly, not a branch ref.

If commits are made in detached state:

- commits are created normally,
- parent chain is valid,
- but no branch pointer advances.

These commits can become unreachable once HEAD moves away.

Recovery:

- create a branch ref pointing to that detached commit hash (write hash into .pes/refs/heads/) before GC.

Q6.1: Algorithm for garbage collection of unreachable objects

Use mark-and-sweep.

Mark phase:

1. Start from all refs in .pes/refs/heads/* (and detached HEAD hash, if any).
2. Traverse commits recursively:
    - mark commit hash,
    - traverse parent,
    - traverse commit tree.
3. Traverse tree objects recursively:
    - mark tree hash,
    - mark blob hashes,
    - recurse into subtree hashes.

Sweep phase:

1. Scan all .pes/objects// files.
2. Convert path to object hash.
3. Delete objects not in reachable set.

Data structure:

- hash set for reachable hashes (O(1) expected lookup).
- stack/queue for DFS/BFS traversal.

Scale estimate for 100,000 commits and 50 branches:

- typically near unique reachable history, not 100,000 x 50,
- about 100,000 commit objects plus corresponding reachable trees/blobs,
- usually hundreds of thousands to a few million total reachable objects depending on file churn.

Q6.2: Why concurrent GC with commit is dangerous

Race example:

1. Commit process writes new blobs/trees first.
2. Before commit ref update, these objects are temporarily unreachable from refs.
3. Concurrent GC marks from current refs and misses those new objects.
4. GC sweep deletes them.
5. Commit object is then written/updated to reference deleted objects.

Result: broken commit graph with missing objects.

How Git avoids this:

- uses locking and coordinated updates around refs,

- uses conservative pruning windows (do not immediately prune recent loose objects),
- manages pack/visibility safely to avoid deleting in-flight objects.

5. Notes

- Integration test script path issue with spaces was handled by running from a no-space symlink path:
  - /tmp/pesvcs-nospace/repo
- This preserves provided files unchanged while allowing successful execution.